Software Engineering Assignment 04

LMS Smart Campus Room Availability Checker

Salim Tokhtobayev
Student ID: 24B070520

This report documents the extension of a working one-room prototype into a four-room availability checker. The submission contains the initial and final source, executable tests, recorded results and AI-use evidence. The original 16 regression checks pass in both versions.

Task 01 Codebase analysis

The Practice 06 source was unavailable, so an equivalent minimal prototype was reconstructed, as permitted on assignment slide 9. It was run and tested before the final version was implemented. Initial map: index.html loads app.mjs and styles.css. app.mjs contains the room data, checkAvailability(), mount() and preview().

Risk and source	Evidence and planned improvement
Repeated validation initial/app.mjs:9-16 and 40-45	checkAvailability() and preview() repeat format, order and duration rules. preview() omits the 09:00 boundary. Use a single shared time validator.
Data duplicated in markup initial/app.mjs:2-6 and 29-30	Room data is declared as an object and repeated as literal HTML. Render details and bookings from the selected room object.
Mixed responsibilities initial/app.mjs:25-59, mount()	Data presentation, event handling and feedback changes are coupled. Separate data, rules, filtering, state and UI.
Stale feedback metadata initial/app.mjs:38-48 and 55-57	preview() updates text but retains the previous data-code and aria-invalid values. Use a single render path that refreshes all feedback.

Prioritized plan recorded before final implementation

1. Preserve the initial source and run the original-rule regression suite.
2. Extract pure time and availability rules; add the room catalog and filter predicates.
3. Add selection state and shared rendering; run regression, integration and browser checks.

Modified AI advice: the reviewer recommended recalculating on every room change. I applied this after the first explicit check. Before that check, room changes preserve the ready state. This avoids presenting an unrequested result while still removing stale success after a completed check.

Task 02 Feature extension

The final application is named LMS and uses a minimal blue interface. Its four fictional rooms each have an ID, name, capacity, equipment list, blocked status and separate booking array. The demonstration day is fixed at 2 November 2026, 09:00 in Almaty.

Room	Capacity and equipment	Bookings and status
R101 Seminar Room	20; Projector, Whiteboard	10:00 - 11:00; 14:00 - 15:00. Not blocked.
R202 Computer Lab	30; Projector, Whiteboard, Computers	11:00 - 12:30. Not blocked.
R303 Study Room	8; Whiteboard	13:00 - 14:00. Not blocked.
R404 Lecture Hall	60; Projector, Whiteboard, Microphone	09:30 - 10:30. Blocked for maintenance.

Capacity uses room.capacity >= attendees. Equipment uses every(), so a room must contain all selected items. Blocked rooms may match the requirements but are visibly marked and can never return available. When filters remove the selected room, the first remaining match is selected; no matches clears selection and disables checking.

Demonstration	Observed result
25 attendees + Projector + Computers	Only R202 matches. At 12:30 - 13:30 its booking is only touched, so the interval is available.
Computers + Microphone, or 61 attendees	No matching rooms; guidance is shown and the check button is disabled.
R101, 11:00 - 12:00; then select R202	Available changes immediately to overlap. R202 uses its own 11:00 - 12:30 booking.
Select R404, 11:00 - 12:00	Unavailable because blocked, even outside its existing booking.

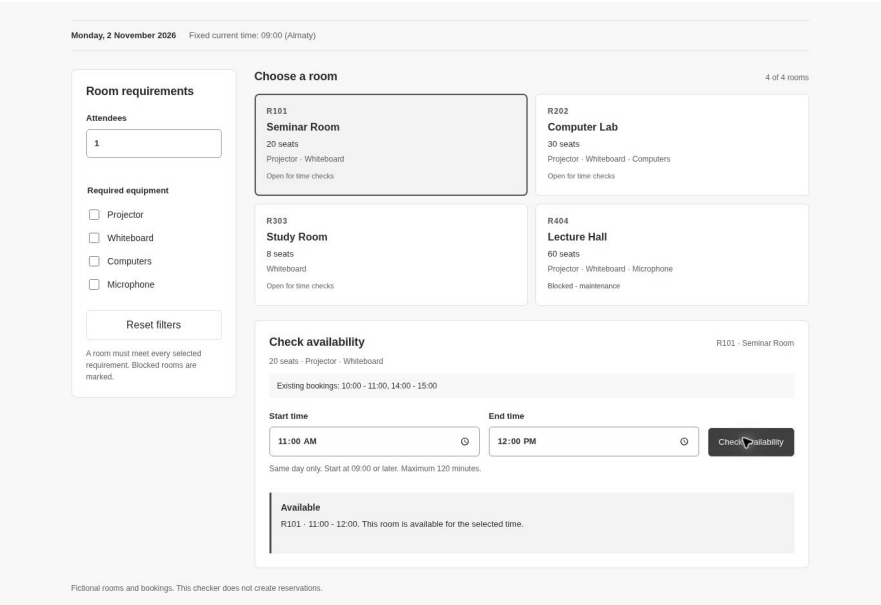
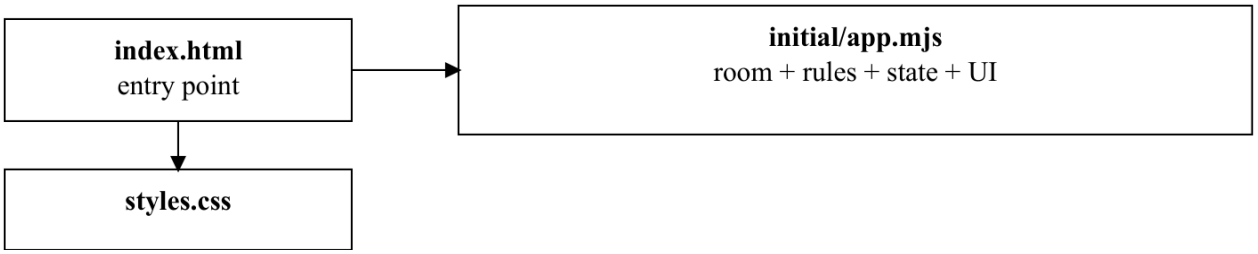


Figure 1. Actual browser capture of the final catalog and successful R101 check, reproduced in monochrome.

Task 03 Refactoring

Before: a single JavaScript module mixed room data, availability, duplicated hints, DOM creation and event handling. After: six modules have explicit responsibilities. HTML and CSS remain presentation assets.

Before



After

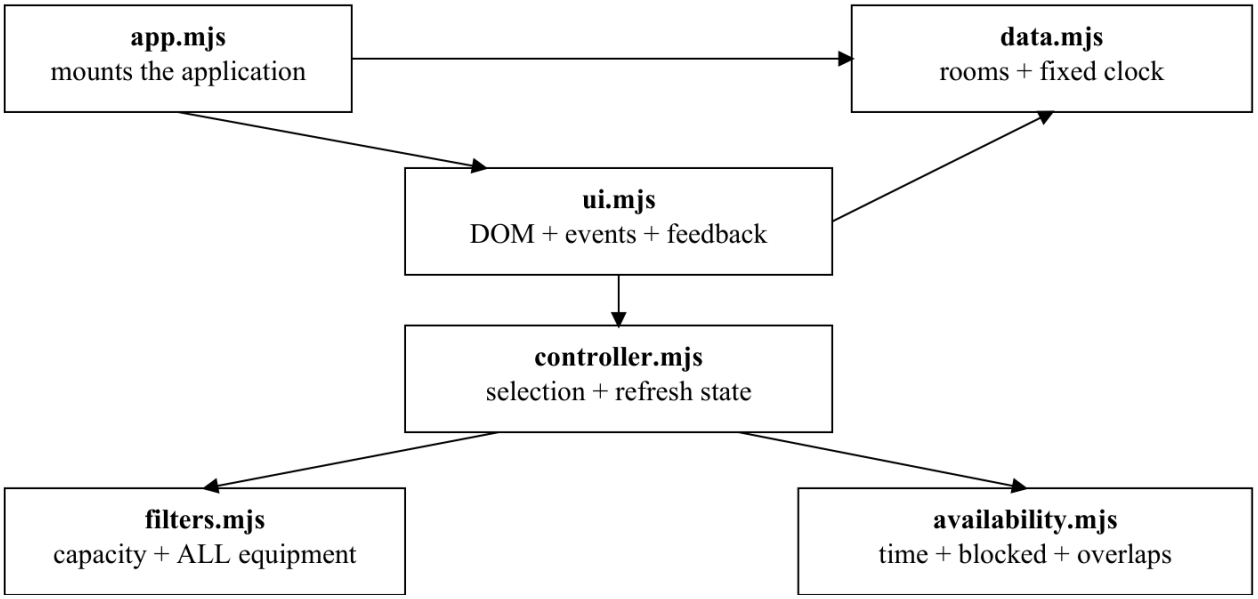


Figure 2. Primary module dependencies. app.mjs imports data and UI; UI calls the controller and reads display constants. availability.mjs also imports NOW from data.mjs. Filtering and availability never access the DOM.

Final module	Single responsibility
data.mjs	Four deeply frozen room objects and demonstration constants.
availability.mjs	toMinutes(), validateTimeWindow(), checkAvailability().
filters.mjs	validateAttendees() and filterRooms().
controller.mjs	Selection by ID, criteria and interval state; computes a current view.
ui.mjs and app.mjs	Render accessible controls and feedback; wire events; mount once.

Design decision 1. Pure functions receive room data rather than reading the screen. The same 16 original checks can run against both versions without a browser, making behavioral preservation measurable.

Design decision 2. The controller owns selection and recalculation; the UI has one render() path. Empty filters, room changes and corrected times therefore update details, result codes and error attributes together. No persistence is needed for this fixed-day checker.

Task 04 Automated testing

The tests were executed, not estimated. Both Node suites use node:test and strict assertions with no external test dependencies. The browser suite mounts the actual final UI and dispatches DOM events. Its final focus case exposes the fixture before asserting focus.

Requirement	Executable evidence
AC1 Valid time	Malformed, missing and impossible times; 08:59 vs 09:00; equal and reversed endpoints.
AC2 Maximum duration	11:00 - 13:00 accepted; 11:00 - 13:01 rejected on an empty fixture.
AC3 Blocked room	Blocked fixture returns blocked even with an empty booking array.
AC4 Overlap	Touching both endpoints accepted; one-minute overlap, identical and enclosing intervals rejected.
AC5 Immutable data	Unsorted room and booking fixtures are frozen; output is checked and serialized data compared before and after.
F1 Capacity	Exact capacity included; one seat too many excluded; invalid counts rejected.
F2 All equipment	Projector + Computers yields only R202; Computers + Microphone yields none.
F3 Blocked result	Catalog, controller and rendered DOM checks all reject a blocked room.
F4 Empty state	Controller clears selection; actual UI shows guidance and disables checking. Reset recovers.
F5 Room switching	R101 success becomes R202 overlap for the same interval without a second submit.
AC6 Responsive feedback	Real DOM result/error checks, keyboard Enter and a narrow iframe layout inspection.

Commands and actual results

From the extracted submission directory:
node --test --test-reporter=tap tests/initial.test.mjs
node --test --test-reporter=tap tests/final.test.mjs
node --test

Suite	Passed / total	Recorded output
Initial Node regression	16 / 16; 0 failed	results/initial-tests.txt
Final Node tests	35 / 35; 0 failed	results/final-tests.txt
Combined Node discovery	51 / 51; 0 failed	Same two suites run together; no additional test cases.
Browser real DOM tests	9 / 9; 0 failed	results/browser-tests.txt

For browser checks: start python3 -m http.server 8000, then open http://localhost:8000/tests/ui.test.html. Each page load reruns the nine tests. Execution used Node.js v24.22.1 and Chrome. Node output contains the original TAP pass/fail results and durations.

The final Node suite consists of 16 original-rule tests, 10 catalog/filter tests and 9 controller integration tests. The nine browser checks are separate. No coverage percentage or speedup is inferred from these counts.

Task 05 Quality evaluation

Measure	Initial	Final	How measured
Original regression checks	16 / 16	16 / 16	The same regression() cases are imported by each Node suite.
Full Node suite	16 / 16	35 / 35	Actual outputs in results/. Final adds 19 extension/integration cases.
Real DOM UI tests	Not run	9 / 9	tests/ui-browser.mjs and browser-tests.txt. Not counted as initial failures.
Duplicate validation locations beyond one canonical implementation	1	0	Initial checkAvailability() and preview() repeat format/order/duration checks. Final validateTimeWindow() owns these rules. One duplicate location removed.
Application JS modules	1 mixed module	6 focused modules	Count .mjs files under initial/ and final/src/; responsibilities are shown in Figure 2.

Remaining limitations. Both versions use fictional, fixed-day in-memory data. The final version does not create bookings, support overnight intervals or handle concurrent users. Native time-input appearance depends on the browser. Responsive inspection used a 390px-wide iframe with a 375 CSS px inner viewport, not a physical phone or full screen-reader audit.

Task 06 AI reflection

I provided the AI assistant with the repository map, the original acceptance criteria and the limits of the assignment. I explained that the Practice 06 source was unavailable (because limits of bolt.new and I used Codex), so the starting point was a reconstructed one-room checker. I also specified a fixed current time, fictional bookings, four rooms, capacity and equipment filters, and no database or login.

The most useful recommendations were to extract pure availability rules and separate selection state from rendering. The review identified duplicated validation in checkAvailability() and preview(), as well as hardcoded room details. It also noticed that editing time could leave old feedback attributes behind. I modified the recommendation to recalculate on every room change: before the first explicit check, the interface remains ready; after a check, every relevant change refreshes the result.

I verified the rules using explicit boundary examples and executable assertions. These included exactly 120 and 121 minutes, both touching endpoints, a one-minute overlap, blocked rooms and frozen booking data. I also checked the actual browser behavior for successful filtering, an empty result and a change from an available room to a conflicting room.

Tests influenced the design by making pure functions and one feedback path useful, rather than merely stylistic. The same regression cases pass against both versions, while integration checks cover selection and recalculation together. Next, I would add wider browser and assistive-technology testing. I would introduce real dates or reservations only if the product requirements expanded.

Appendix AI use and verification

Tool and model: ChatGPT / Codex, GPT 6 Sol

Date: 02.11.2026
Role: baseline review, implementation assistance, test design.

Selected prompt excerpts

"Act as the AI reviewer for Software Engineering Assignment 04. Read only submission/initial/app.mjs and initial/index.html and initial/styles.css. Repository map: initial/app.mjs contains room data, checkAvailability, mount and preview; index.html mounts it; styles.css handles presentation."

"Before any final implementation, identify >=3 maintainability risks with exact function references and give a short prioritized plan. Explain tradeoffs of including blocked matching rooms versus hiding them, and localStorage versus no persistence. Recommend tests including an integration check. Return concise recommendations only, do not edit files."

The same prompt supplied AC1-AC6, all five new filtering rules, the four-room schema, the fixed 09:00 clock and the prohibition on login, APIs, a database, persistence and reservation creation. These were the constraints used to judge the response.

Selected actual review output

"Both define time parsing and validation, but preview() omits the 09:00 boundary. Extract shared parsing and interval validation so hints and submitted results cannot drift."

"Room identity, capacity, equipment and bookings are embedded in HTML, while submission separately uses the exported room."

"Centralize feedback updates and recompute availability whenever room selection changes. Define what happens when filters exclude the selected room, and show an explicit empty state."

"No persistence keeps this fixed-date prototype deterministic and meets the stated scope. Recommend no persistence."

Decisions and verification record

Output used or modified	Decision and independent evidence
Shared rules and separate modules	Implemented availability.mjs, filters.mjs, controller.mjs and ui.mjs. Both versions pass the same 16 regression cases.
Always recalculate on room change	Modified to preserve idle feedback until the first explicit check. Dedicated controller test covers that choice; after checking, UI test covers automatic refresh.
Include blocked matching rooms	Accepted with an explicit maintenance label. Unit, integration and DOM tests confirm that a matching blocked room is never available.
No localStorage	Accepted. Data is read-only and resets on reload; source contains no persistence code.

Problems found: duplicated validation, hardcoded data, mixed responsibilities and stale result/error attributes in the initial version. The AI review was static and explicitly stated that it had not run tests. Its conclusions were therefore checked against source and the actual executed suites rather than treated as proof.

Verification sources: supplied Assignment 04 slides 9-19 and 26; initial/app.mjs; final/src/; tests/regression.mjs; tests/final.test.mjs; tests/ui-browser.mjs; and all three files in results/. AI-generated implementation was checked through executable assertions and observed browser outcomes. No fabricated test metrics or simulated review transcript is used.